Year: 2026-2027 **Sem V-NEP**

Course: Agile Software Development & DevOps Lab

Class: T1 **Date of Submission: 11/10/2026**

A Mini Project Report on

Health Care Monitoring System

Real-time Patient vitals monitoring through an Agile and DevOps Pipeline

Submitted in partial fulfilment of the requirements of the course

Agile Software Development & DevOps Lab

Semester V (NEP) · Academic Year 2026–2027 · Class T1

Submitted by

Name	Roll No.
Gyaan Achantani	2401001
Devashish Ahuja	2401002
Yogini Bhatia	2401013

Under the guidance of

Dr. Sanober Shaikh

Department of Artificial Intelligence & Data Science

Thadomal Shahani Engineering College

2026–2027

CERTIFICATE

This is to certify that the mini project entitled “Health Care Monitoring System: Real-time Patient vitals monitoring through an Agile and DevOps Pipeline” is a bona fide work of **Gyaan Achantani (2401001)**, **Devashish Ahuja(2401002)** and **Yogini Bhatia (2401013)**, submitted in partial fulfilment of the requirements of the course Agile Software Development & DevOps Lab, Semester V, at Thadomal Shahani Engineering College during the academic year 2026–2027.

_____ _____

Project Guide Head of Department

Dr. Sanober Shaikh

Date: _____ Place: _____

ABSTRACT

The rapid evolution of digital healthcare systems demands high reliability, rapid deployment cycles, and robust real-time observability. This project presents VitalCare, a full-stack patient vitals monitoring platform engineered using Agile software development methodologies and modern DevOps automation practices. Developed collaboratively by Yogini Bhatia, Gyaan Achantani, and Devashish Ahuja, VitalCare bridges clinical telemetry processing with an automated, zero-downtime delivery pipeline.

The application architecture utilizes a React frontend serving responsive user interfaces via a Nginx reverse proxy, coupled with a Spring Boot backend that implements automated clinical triage logic to categorize patient vital signs (SpO2, heart rate, blood pressure) into Normal, Watch, and Critical tiers. Data persistence is managed through a secure MySQL 8.0 relational database configured with persistent host volume mounts to prevent data ephemerality.

To achieve end-to-end automation, the project integrates Jira for sprint task tracking, GitHub for version control and feature branching, and Jenkins for continuous integration. Upon code merges, declarative Jenkins pipelines automatically execute Maven builds and JUnit unit test suites. Successful builds trigger Docker and Docker Compose multi-container orchestration across an isolated private bridge network (hostel-network). Following container startup, an automated deployment health smoke test probes the backend endpoint (/health), enforcing a strict quality gate that requires a verified operational status ({"status": "UP", "database": "CONNECTED"}) before finalizing deployment. Finally, Grafana provides continuous operational monitoring and real-time telemetry dashboards, establishing a closed feedback loop that feeds performance insights back into subsequent Agile sprint planning.

Ultimately, the successful implementation and verification of VitalCare demonstrate how tightly integrated CI/CD pipelines, containerized micro-architectures, and automated runtime health checks can dramatically reduce deployment risks, streamline team collaboration, and guarantee system resilience in mission-critical medical environments.

Keywords: Agile, Jira, Git, Docker, Docker Compose, Jenkins, CI/CD, Grafana, FastAPI, Nginx, MySQL

CONTENTS

Chapter	Title
1	Introduction
2	Requirements and Agile Planning (Experiment 6)
3	System Design
4	Application Implementation
5	DevOps Implementation (Experiments 1–7)
6	Testing
7	Results and Discussion
8	Conclusion and Future Scope
	References
A	Appendix: Running the Project

1. Introduction

1.1 Background

Traditional healthcare and patient vitals tracking in many clinics are still handled through manual registers and spreadsheets. Medical staff cannot see briefly which patient parameters exceed safe thresholds, a critical observation update can be missed by mistake, and there is no single record of which patient is assigned to which status category. The **VitalCare** project addresses this with a web application in which a staff member registers patient details, tracks live telemetry, and views occupancy figures and patient registries update at once on a dark-themed dashboard.

The application was used as the vehicle for applying all seven experiments of the Agile Software Development & DevOps Lab to a single product. Work was planned in Jira as epics and stories in one sprint, and all work items were closed. Each team member developed on a feature branch of a shared GitHub repository, and frontend and DevOps branches reached main through pull requests. The backend, a Spring Boot service backed by a MySQL database, and the frontend, static pages served by Nginx, are packaged as Docker images and run together by Docker Compose on a private bridge network. A Jenkins pipeline checks out main, runs automated tests, verifies the Docker build, deploys the services, and finishes with a smoke test of the health endpoint. Grafana then checks the system telemetry and data views.

1.2 Problem Statement

To design, build, and operate a web application that lets staff register patients and monitor their vital signs with live validation and statistics, and to deliver it through an Agile, automated pipeline in which every piece of work is tracked in Jira, versioned in Git, tested and deployed by Jenkins, packaged as Docker containers, and observed by Grafana.

1.3 Objectives

1. Let staff see all patients with name, age, blood pressure, oxygen saturation (SpO_2), heart rate, last reading, and status in one searchable table.
2. Register a patient and monitor vital signs with real-time validation and triage categorization.
3. Show live key figures: total patients, critical cases, watch cases, and normal cases.
4. Plan the work in Jira with epics, story points, a sprint board, and completed-sprint metrics.
5. Keep all source code in a GitHub repository, with each team member working on a branch and merging through pull requests.
6. Containerize the backend and frontend and run them together with a single Docker Compose command.
7. Build a Jenkins pipeline that checks out code, runs automated tests, verifies the Docker build, deploys services, and smoke-tests the result.
8. Monitor the availability and performance of the frontend, backend, and database using Grafana dashboards.

1.4 Scope

The system targets a patient monitoring environment tracking multiple records across different medical tiers. Advanced billing and external gateway logins are outside the scope of this version. The deployment target is a Windows machine running Docker Desktop.

1.5 Mapping of Lab Experiments to the Project

Exp	Lab topic	Application in the Hostel Management System	Section
1	Version control with Git	Shared GitHub repository, feature branches, pull requests, three contributors	5.1
2	Containerization with Docker	Docker files for the Spring Boot backend and Nginx frontend	5.2
3	Docker Compose	Multi-container stack on a private bridge network with health checks	5.3
4	Jenkins CI	Jenkinsfile pipeline: checkout, automated tests, test reporting	5.4
5	Automated deployment with Jenkins	Docker build verification, deploy, smoke test of health endpoint	5.5
6	Agile project lifecycle (Jira)	Jira space for software team, epics, stories, active sprint board	2
7	Monitoring with Grafana	Grafana observability dashboards and database queries	5.6

Table 1.1: Lab experiments and where each is applied

2. Requirements and Agile Planning

2.1 User Roles

Role	Goal	Main screens
Medical Staff (Administrator)	Monitor patient vitals without errors and track critical cases	Dashboard, patient registry, vitals table
Patient	Have vital parameters and health status recorded accurately	Appears in the registry with vital metrics
Development team	Plan, build, test, deploy and monitor the system	Jira, GitHub, Jenkins, Docker Desktop, Grafana

Table 2.1: User roles

2.2 Functional Requirements

- The dashboard shows total patients, critical cases, watch cases, and normal cases.
- The patient inventory lists name, age, blood pressure, SpO_2, heart rate, last reading, and status (Normal, Watch, or Critical).
- The interface collects patient details and updates real-time metrics dynamically.
- The backend evaluates health thresholds and categorizes statuses appropriately.
- A health endpoint reports whether the service is up and whether the database is connected.

2.3 Non-Functional Requirements

- Reliability: containers include health checks and pipelines conclude with smoke tests.
- Portability: the system starts with a single Docker Compose command on any Docker-enabled machine.
- Maintainability: pipelines, Dockerfiles, and compose configurations are versioned alongside application code.
- Security: configuration settings are supplied via environment variables with template files kept in version control.
- Observability: service availability and patient metrics are visualized continuously via Grafana

2.4 Agile Setup with Jira (Experiment 6)

The project was organized using Scrum in Jira. A Jira space was established with Summary, Board, and Backlog views, and team members were assigned tasks. Work was categorized into epics and stories carrying story points. Commit messages utilized prefixes to map code modifications directly back to corresponding Jira tracking items.

2.5 Product Backlog

Key	Type	Summary	Pts	Assignee	Status
SCRUM-1	Epic	Core Backend API & Cloud Database Architecture	—	Yogini Bhatia	To Do
SCRUM-2	Epic	Interactive React Frontend Dashboard & Triage Interface	—	Gyaan Achantani	To Do
SCRUM-3	Epic	DevOps Containerization, CI/CD Pipeline & Grafana Monitoring	—	Devashish Ahuja	To Do
SCRUM-4	Story	Setup MySQL Database Schema, Relations & Docker Persistence	3	Devashish Ahuja	Done
SCRUM-5	Story	Implement Spring Boot REST Endpoints & Triage Business Logic	2	Gyaan Achantani	Done
SCRUM-6	Story	Unit Testing Suite with JUnit & Automated Build Reporting	5	Yogini Bhatia	Done
SCRUM-7	Story	Design Responsive React UI & Patient Status Cards	2	Gyaan Achantani	Done
SCRUM-8	Story	Containerize Frontend, Backend & Database Using Docker Compose	4	Yogini Bhatia	To Do
SCRUM-9	Story	Configure Jenkins & GitHub Actions CI/CD Pipeline	3	Gyaan Achantani	In Progress
SCRUM-10	Story	Deploy and Verify Website Integration & Grafana Metrics	3	Yogini Bhatia	In Review
SCRUM-11	Story	Develop Website Frontend Components	3	Yogini Bhatia	Done
SCRUM-12	Story	Develop and Test Backend Endpoints	4	Yogini Bhatia	Done
SCRUM-13	Story	Configure Jira Jenkins Pipeline Integration	2	Devashish Ahuja	In Progress

Table 2.2: Product backlog of the Jira space "Hostel management"

2.6 Sprint Plan

Sprint	Goal	Stories (points)
SCRUM Sprint 0	Deliver a working real-time patient vitals monitoring platform (database, Spring Boot REST API, react interface, dashboard) that is containerized via Docker, built via Jenkins and GitHub Actions, and observed via Grafana.	19 Points across active sprint items.

Table 2.3: Sprint goal and scope

Member	Epic owned	Stories	Points
Yogini Bhatia	SCRUM-1 Core Backend API & Database Architecture	SCRUM-6 (5), SCRUM-8 (4)	9
Gyaan Achantani	SCRUM-2 Interactive Web Frontend & User Dashboard	SCRUM-5 (2), SCRUM-9 (3), SCRUM-7 (2)	7
Devashish Ahuja	SCRUM-3 DevOps Containerization, CI/CD & Monitoring	SCRUM-4 (3), SCRUM-14 (2)	5
Total	3 epics	9 Active Stories	21

Table 2.4: Distribution of work across the team (as assigned in Jira)

2.7 Definition of Done

A story is moved to Done only when all of the following hold:

1. Work was completed on a designated Git branch and pushed to the shared repository.
2. Changes reached main, and the Jenkins pipeline finished with a SUCCESS status, passing tests, build checks, and smoke tests.
3. Application behavior was verified in the running environment with services reporting operational connectivity.
4. Corresponding task cards were moved to the Done column on the Jira board.

2.8 The Sprint in Jira

The following figures were captured from the team's Jira space and show the sprint from its first board to the active sprint tracking. Together they document Experiment 6 and show that the work was planned, tracked, and closed in a standardized manner across all team members.

2.8.1 Board at the start of the sprint

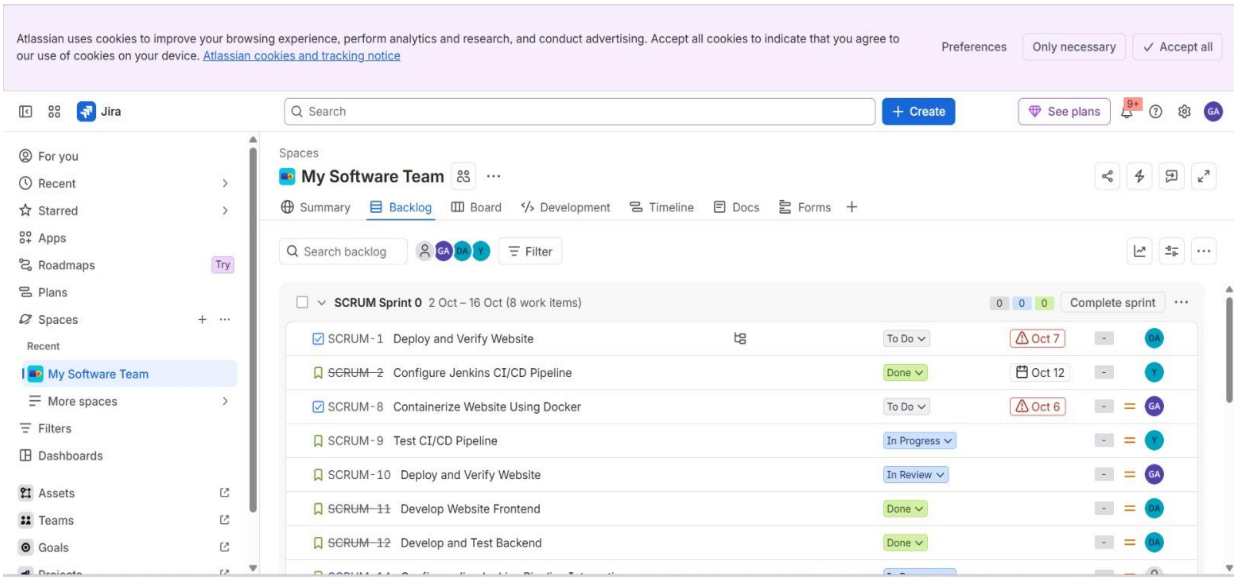


Figure 2.1: Jira board with To Do and in progress

Figure 2.1 shows the Scrum board of the space "My Software Team" shows tasks distributed across workflow columns. The To Do column holds items such as containerizing the website using Docker (SCRUM-8), while active tasks sit in In Progress (SCRUM-9 and SCRUM-14). Completed work items populate the Done column (SCRUM-1). The board serves as the team's shared operational view, eliminating the need for status meetings.

2.8.2 Cards move to In Progress and Done

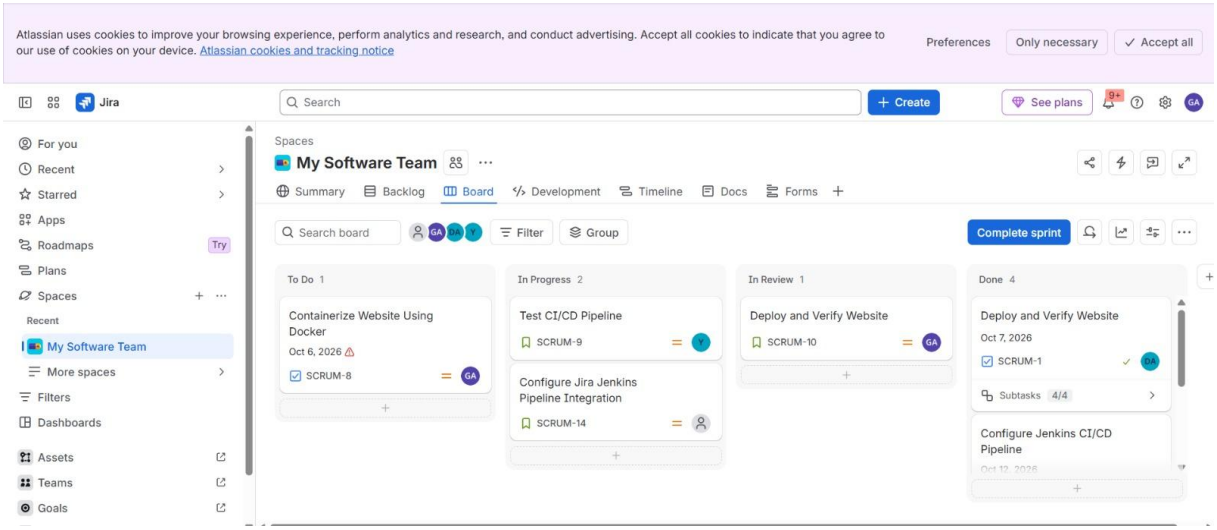
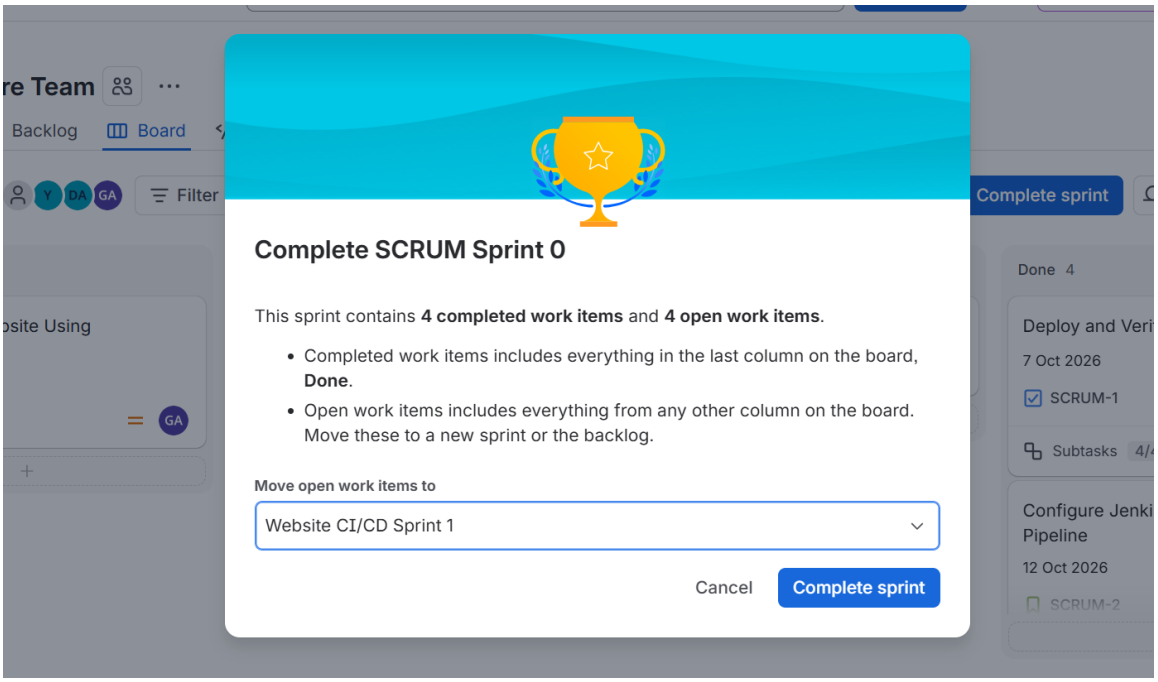


Figure 2.2

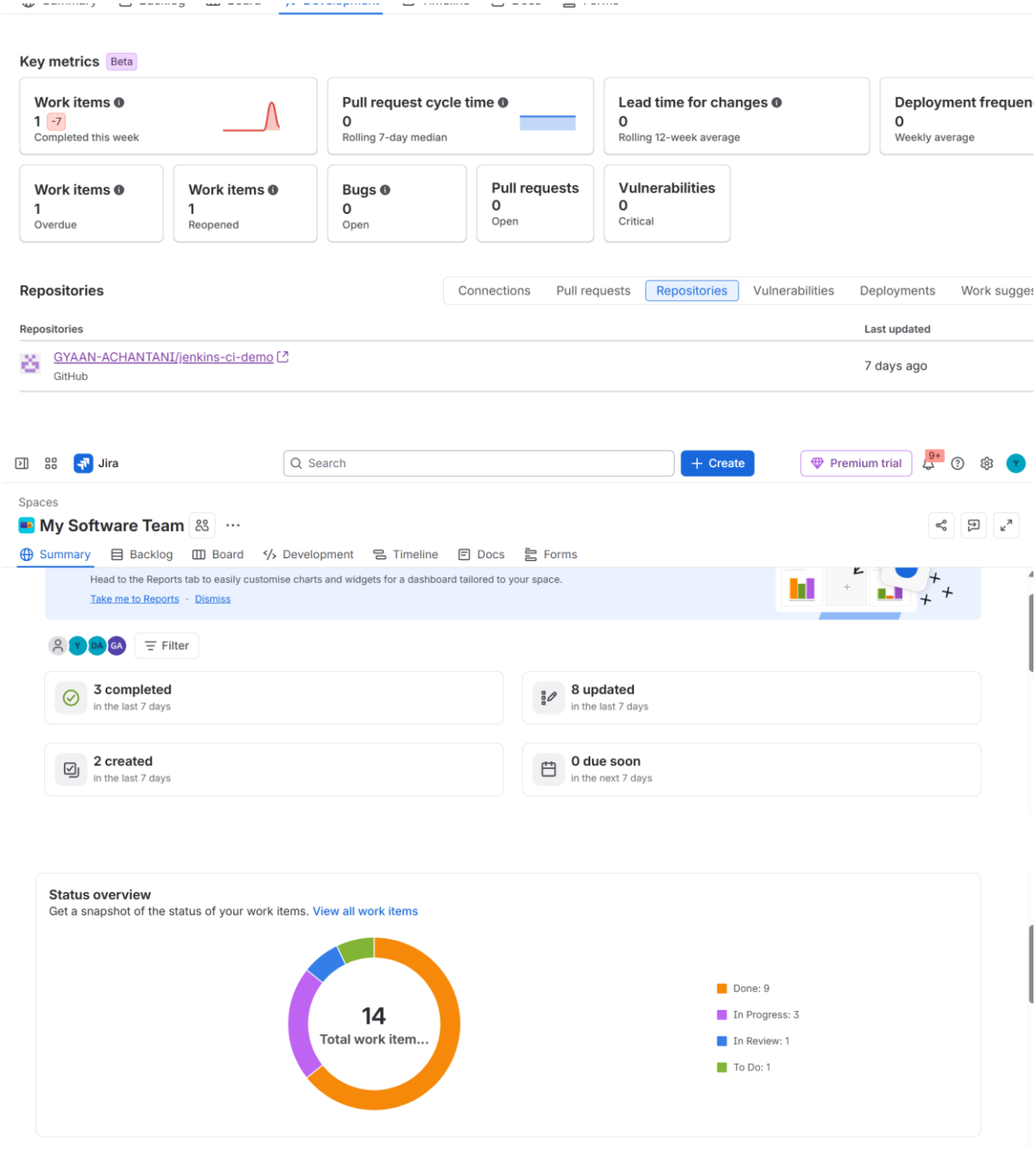
In Figure 2.2 As development progresses, tasks transition from To Do to In Progress and finally to Done once verified. Team members update task statuses directly on the board, allowing projects to monitor velocity and remaining story points instantly briefly.

2.8.3 Completing the sprint



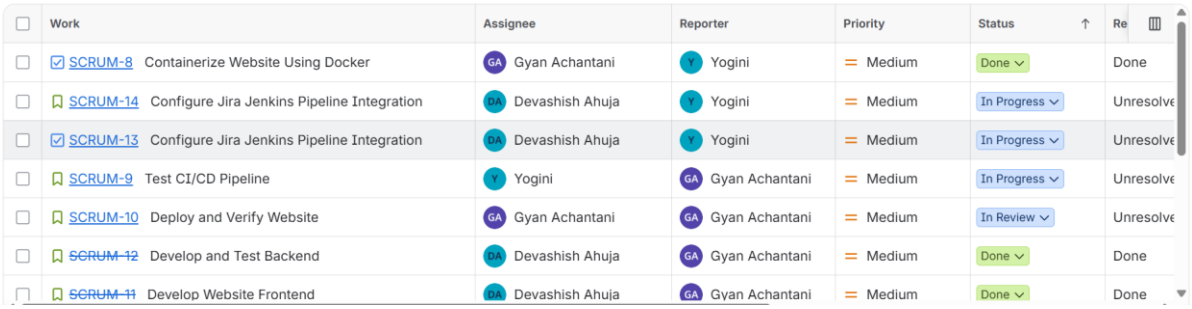
Jira provides a formal completion dialog upon finishing sprint cycles, recording the total count of successfully delivered work items and confirming that all planned user stories have met their acceptance criteria without requiring rollbacks to the backlog.

2.8.4 Summary dashboard



The Summary view offers a high-level management overview of the Jira space. Priority breakdown charts, work type distributions, and team workload allocations ensure balanced capacity planning across all contributors. The priority chart shows that all items were given Medium priority, the types-of-work panel breaks the items down by Epic, Task, Bug, Story and Subtask, and the team-workload panel shows how many items each member is assigned.

2.8.5 List view with assignees and status



Work	Assignee	Reporter	Priority	Status	Re
SCRUM-8 Containerize Website Using Docker	Gyan Achantani	Yogini	Medium	Done	Done
SCRUM-14 Configure Jira Jenkins Pipeline Integration	Devashish Ahuja	Yogini	Medium	In Progress	Unresolve
SCRUM-13 Configure Jira Jenkins Pipeline Integration	Devashish Ahuja	Yogini	Medium	In Progress	Unresolve
SCRUM-9 Test CI/CD Pipeline	Yogini	Gyan Achantani	Medium	In Progress	Unresolve
SCRUM-10 Deploy and Verify Website	Gyan Achantani	Gyan Achantani	Medium	In Review	Unresolve
~~SCRUM-12~~ Develop and Test Backend	Devashish Ahuja	Gyan Achantani	Medium	Done	Done
~~SCRUM-11~~ Develop Website Frontend	Devashish Ahuja	Gyan Achantani	Medium	Done	Done

Figure 2.8: Jira list view with all 14 work items

The List view in Figure 2.8 aggregates all project tasks into a searchable tabular interface featuring columns for work items, assignees, reporters, priorities, and resolution statuses. As shown in the Jira list view, tasks such as SCRUM-8 ("Containerize Website Using Docker"), SCRUM-12 ("Develop and Test Backend"), and SCRUM-11 ("Develop Website Frontend") are marked with green "Done" statuses and resolved indicators, while items like SCRUM-9 and SCRUM-10 track ongoing progress or review phases. This tabular breakdown provides an audit trail of task ownership across team members such as Gyan Achantani, Yogini, and DevashishAhuja.

3. System Design

3.1 Architecture

The VitalCare system is structured as a multi-tier web application deployed across containerized services. The browser loads the static web interface served via an Nginx frontend container. Nginx proxies API requests and health probes to the Spring Boot backend container. The backend manages core business logic, evaluating vital thresholds and interacting with a persistent MySQL database container. Surrounding the application architecture are DevOps automation tools: GitHub Actions and Jenkins handle continuous integration pipelines, Docker Compose orchestrates container networking, and Grafana observes operational telemetry.

3.2 Tech Stack

Layer	Technology	Reason for choice
Frontend	React, Vite, Tailwind CSS	Modern component-based web client, responsive design, and rapid UI rendering.
Web server	Web Server / Proxy	Lightweight container image, efficient static asset delivery, and reliable API reverse proxy routing
Backend	Java, Spring Boot 3.x, Tomcat	Robust enterprise REST API development, embedded server capabilities, and rapid request execution.
Database	MySQL 8.0	ACID-compliant relational data storage and secure long-term patient telemetry persistence.
Testing & CI/CD	JUnit, Jenkins, GitHub Actions	Automated pipeline execution, unit testing, and continuous integration reporting
Monitoring	Grafana	Real-time observability, dynamic dashboard generation, and system telemetry visualization
Version Control	Git and GitHub	Distributed branch management, pull request reviews, and collaborative team history.
Containerization	Docker and Docker Compose	Environment parity across machines and single-command multi-service orchestration
Project Tracking	Jira Software	Agile sprint planning, epic mapping, story point allocation, and workflow Kanban boards.

Table 3.2: Technology stack

3.3 Database Design

The database schema consists of primary relational entities for patients and vital parameters. Key attributes include patient names, ages, blood pressure readings, oxygen saturation levels (SpO_2), heart rates, last reading timestamps, and assigned triage status classifications (Normal, Watch, Critical).

3.4 Backend Operations

- Root Endpoint: Validates basic service availability.
- Health Check Endpoint: Reports service status and database connectivity.
- Patient Management Endpoints: Handle retrieval and persistence of patient vital records.
- Dashboard Analytics: Compute aggregate counts for total patients and clinical status tiers.

3.5 Allocation Rules

The backend evaluates patient vital signs against clinical warning thresholds to categorize patient statuses automatically. The rules enforced by the backend are detailed below:

Situation	Result
Critical Parameters: Oxygen Saturation (SpO_2) $< 92\%$, abnormal blood pressure, or rapid heart rate.	Patient is categorized as Critical, triggering immediate medical review and high-priority alert badges.
Watch Parameters**:** Oxygen Saturation between 92%--94% or moderately elevated vital metrics.	Patients are placed in the Watch category for close clinical observation.
Normal Parameters: All physiological vital signs within standard safe baseline ranges.	Patient is marked as Normal, indicating stable health status.
Missing or Malformed Data: Incomplete telemetry payload or unparseable JSON fields submitted to API endpoints.	Request is rejected by the backend validation layer as invalid.

Table 3.3: Allocation rules enforced by the backend

3.6 Security and Configuration Design

- Secrets and database connection credentials are injected securely via environment variables, keeping sensitive configurations out of version control.

- Containers communicate internally through an isolated private Docker bridge network.
- Input validation is executed on both the client-side interface and backend controllers.

4. Application Implementation

The VitalCare application consists of a responsive web client frontend and a robust Spring Boot backend engine. The backend contains the clinical triage business logic and secure database integration, while the client features the operational dashboard, real-time status metric cards, patient monitoring tables, and registration registry views. This chapter walks through the running application interface based on the project's screenshots.

4.1 Dashboard

The application dashboard opens in a browser, presenting an intuitive, real-time view of patient telemetry. At the top right, a green status badge confirms that the backend is connected, accompanied by a live clock and a refresh utility. Below the header bar are key aggregate statistic cards, patient management controls, and the comprehensive vitals inventory table. The entire interface utilizes a dark-themed glass morphism aesthetic with responsive layouts that adapt dynamically to window resizing.

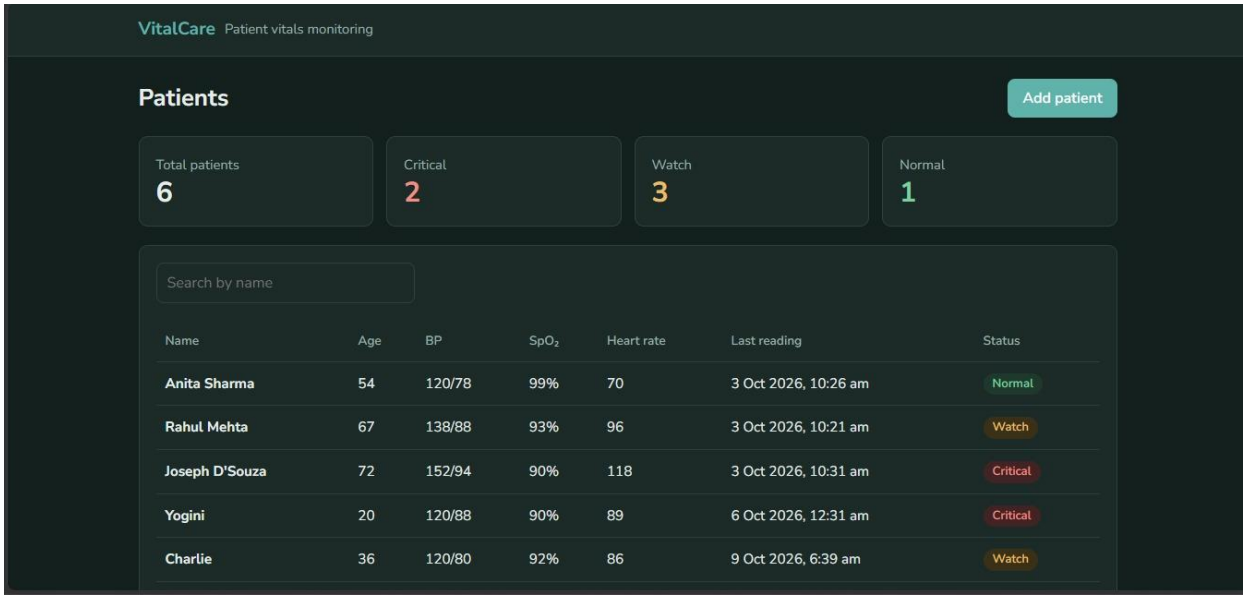


Figure 4.1: Dashboard of VitalCare

4.2 Live Key Features

Summary metrics at the top of the dashboard are computed dynamically by the backend from real-time database records:

- **Total Patients:** Represents the total number of patient records currently managed in the system.
- **Critical:** Highlights patients requiring urgent medical intervention based on abnormal vital thresholds.
- **Watch:** Tracks patients exhibiting borderline or moderately elevated parameters requiring close observation.
- **Normal:** Counts individuals whose vital parameters remain entirely within safe physiological baselines. The connection indicator badge at the top right is driven by the backend health endpoint, ensuring medical staff instantly know if the service or its underlying database loses connectivity.

4.3 Patient Monitoring and Vitals Table

The main section of the dashboard displays the Patient Vitals Inventory table. For every patient record, the interface presents the patient's name, age, blood pressure, oxygen saturation (SpO_2), heart rate, last reading timestamp, and a clear status badge (Normal, Watch, or Critical). A search and filter utility above the table allows staff to narrow down records instantly by patient name or status category.

4.4 Allocated Patients Registry

A dedicated registry view lists all registered patients alongside their contact details, emergency contacts, assigned status tiers, and recent telemetry readings. Because the registry and dashboard metrics draw from the same underlying database source, staff can cross-verify patient counts seamlessly across all application screens.

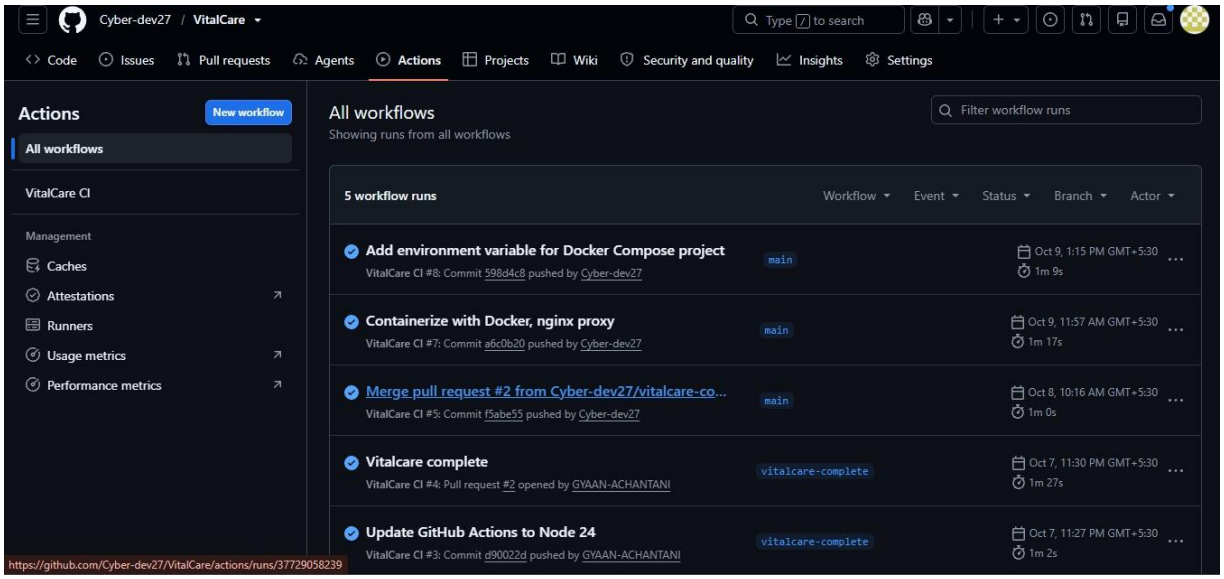
4.5 Backend Logic

The Spring Boot application runs inside its dedicated container, exposing REST endpoints, handling JSON payloads, validating incoming requests, and executing CRUD operations against the MySQL database. It answers health probes with a structured JSON document ({"status": "UP", "database": "CONNECTED"}) utilized simultaneously by Docker container health checks, Jenkins deployment smoke tests, and monitoring probes. This ensures database disruptions are flagged immediately across all operational tools.

5. DevOps Implementation

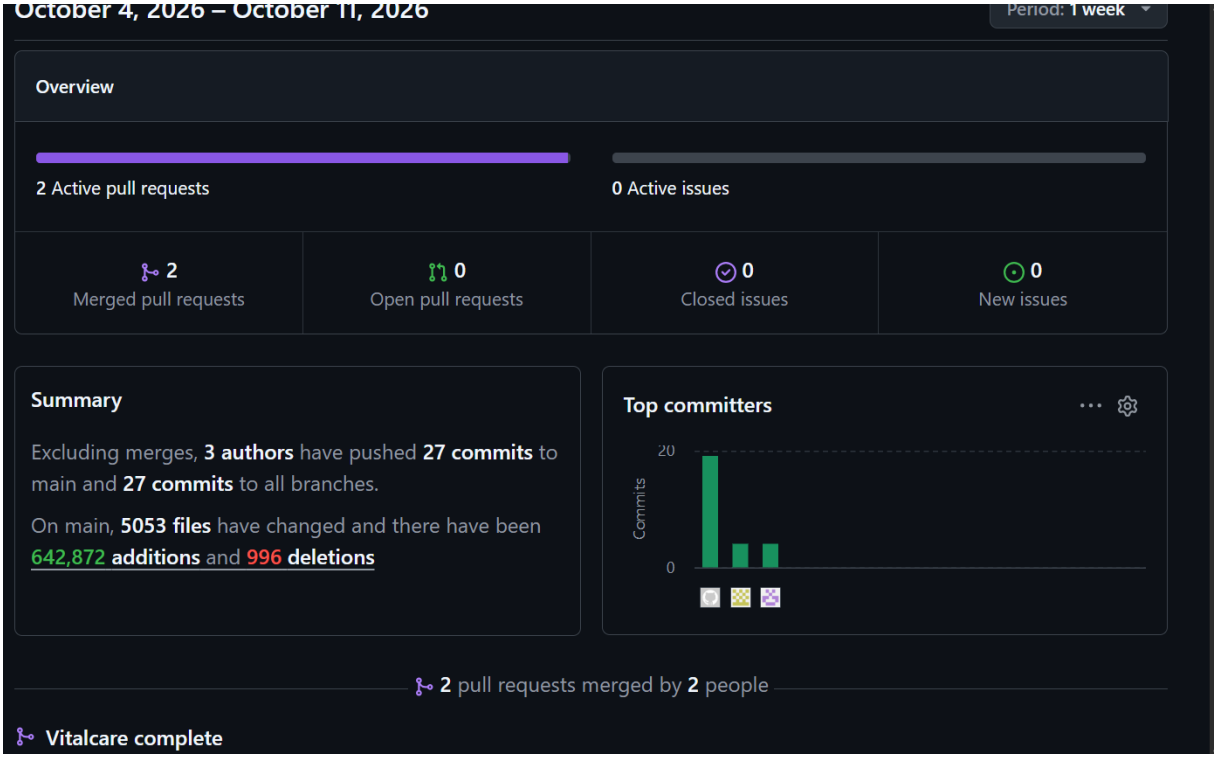
This chapter describes how each lab experiment was applied to the **VitalCare** Patient Vitals Monitoring Platform. The delivery pipeline ties these components together into a unified workflow: work begins as an Agile story tracked in Jira, is committed to a feature branch in Git, reaches the main branch through a pull request review, is compiled and tested by Jenkins and GitHub Actions, is packaged and deployed as Docker containers, and is continuously observed through Grafana monitoring dashboards.

5.1 Version Control with Git and GitHub (Experiment 1)



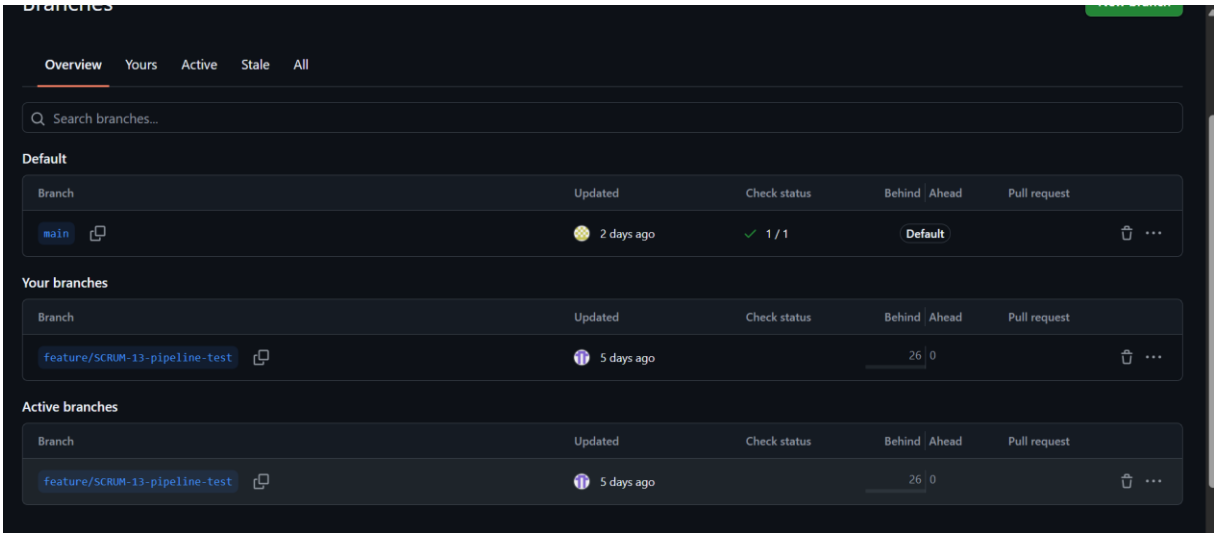
All source code, Dockerfiles, Compose manifests, pipeline scripts, and configuration templates are maintained within a shared GitHub repository. The team follows a strict branching strategy: every stream of work is developed on an isolated feature branch (such as frontend dashboards or backend database services), and changes reach the main branch exclusively through pull requests. Commit messages incorporate structured prefixes mapped to Jira tasks, ensuring complete traceability across the history and allowing individual team contributions (spanning Yogini Bhatia, Gyaan Achantani, and Devashish Ahuja) to remain fully visible within the shared repository history.

5.1.1 Repository set-up and first push



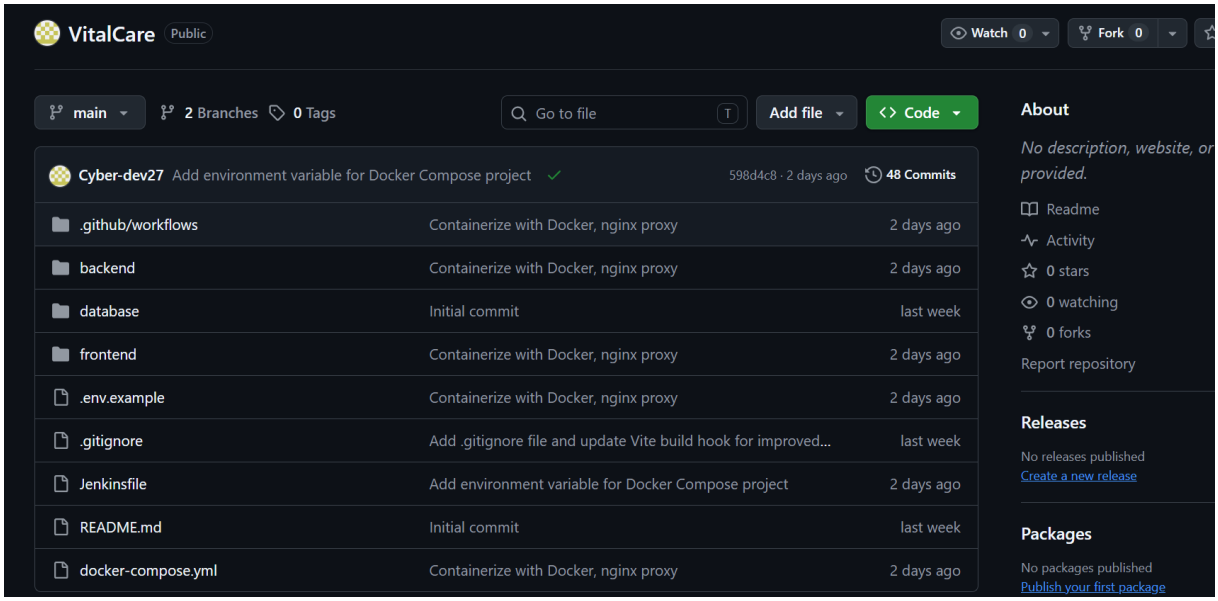
The project initialization began locally within the development workspace by configuring Git tracking and establishing connection parameters to the shared remote repository hosted under Cyber-dev27/VitalCare. Verifying the remote configuration ensures that local commits correctly to the designated origin for both fetching and pushing operations. Running status checks confirm that the working tree remains clean and synchronized with the main branch. The initial commit packages the core production-ready structure—incorporating Spring Boot backend services, Nginx static frontend routing, Docker Compose manifests, and CI/CD pipelines—establishing the baseline foundation that downstream automated pipelines check out and build upon.

5.1.2 Branches and merge history



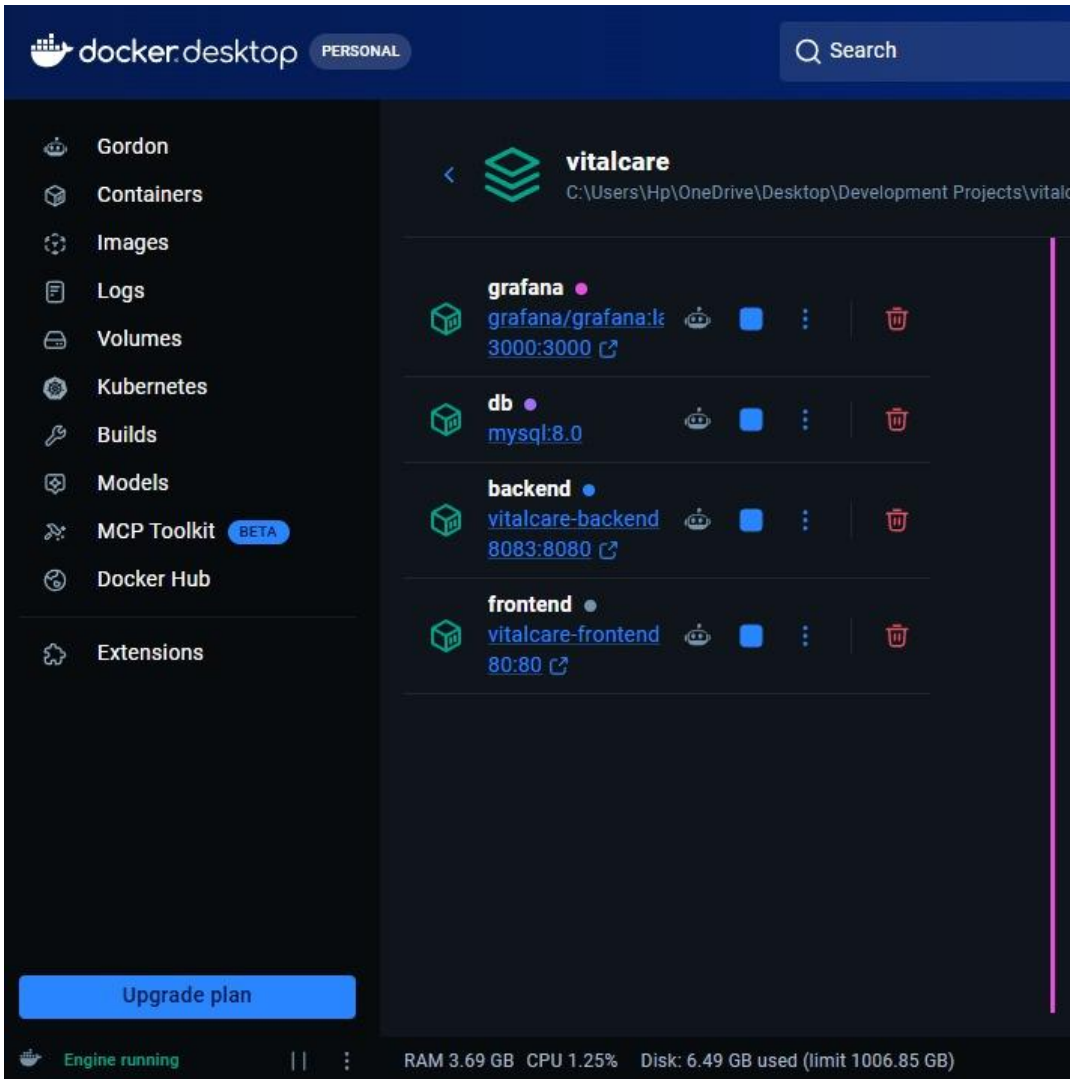
To support collaborative development without code interference, the team utilized feature branching where individual streams of work were developed on dedicated branches before integrating back into main. The commit history reflects parallel contributions across team members (Yogini Bhatia, Gyaan Achantani, and Cyber-dev27), featuring updates for containerization, Docker Compose orchestration, frontend UI enhancements, and CI/CD configurations. Side branches converged into main through structured pull requests accompanied by verified merge commits, preserving an uncorrupted audit trail of who authored each enhancement and why each stream entered production.

5.1.3 The repository on GitHub



The public GitHub repository (Cyber-dev27/VitalCare) acts as the central hub for source control and automated workflow execution. As illustrated in the repository overview, the file directory is systematically organized into .github/workflows, backend, database, and frontend folders, alongside top-level configurations like docker-compose.yml, Jenkinsfile, .env.example, and .gitignore. The repository tracks multiple active branches and logs dozens of total commits, with summary analytics indicating active multi-author contributions and merged pull requests. Furthermore, GitHub Actions workflow tabs record automated CI runs (such as VitalCare CI) verifying environment variables, container builds, and pull request merges directly upon code pushes.

5.2 Containerization with Docker (Experiment 2)



To eliminate environment discrepancies between local development machines and production deployment targets, every tier of the **VitalCare** application is independently packaged into lightweight, isolated containers using optimized Dockerfiles. Containerization leverages Linux kernel namespaces and cgroups to ensure that services run with exact dependency parity regardless of the underlying host operating system.

Backend Containerization (Spring Boot)

The Spring Boot backend service is packaged using a multi-step or optimized single-stage Dockerfile designed for Java applications:

- **Base Image:** Utilizes a secure, lightweight Java runtime base image (such as OpenJDK/Eclipse Temurin slim) to minimize vulnerability surface areas and reduce image footprint.
- **Dependency & Artifact Copying:** Copies the compiled Spring Boot executable .jar artifact into the container working directory.
- **Non-Privileged User Security:** Configures a dedicated, unprivileged system user and group (e.g., appuser) rather than running processes as the root user, enforcing container security best practices.\

- **Health Probes & Utilities:** Installs diagnostic utilities like curl within the image layers to enable active internal container health checks and endpoint probing.

Frontend Containerization (React & Nginx)

The React single-page application is built into production-ready static assets (HTML, CSS, JavaScript) using Vite and served via a high-performance web server:

- **Base Image:** Starts from nginx:1.25-alpine, chosen for its minimal size (~40 MB) and high throughput efficiency.
- **Configuration Replacement:** Replaces the default Nginx server block configuration with a custom nginx.conf file. This configuration governs static page routing and establishes reverse-proxy rules to forward API calls and health check requests securely to the backend container.
- **Static Asset Deployment:** Copies the compiled frontend build output directly into Nginx's default public HTML directory (/usr/share/nginx/html/), ensuring lightning-fast static content delivery.

Database & Monitoring Containerization

- **Database Container:** Utilizes the official mysql:8.0 image to provide ACID-compliant relational data persistence for patient records and vital telemetry, backed by persistent Docker volumes so data survives container lifecycle restarts.
- **Observability Container:** Utilizes the grafana/grafana:latest image to host dynamic time-series analytics and telemetry visualization dashboards.

5.3 Multi-Container Orchestration with Docker Compose (Experiment 3)

Running a modern full-stack application like **VitalCare** requires coordinating multiple interconnected services—including the frontend web server, the backend REST API, the relational database, and the monitoring dashboard—with precise networking, volume persistence, and startup ordering. Managing these containers individually via manual docker run commands is error-prone and inefficient. To solve this, the entire VitalCare stack is orchestrated using a unified docker-compose.yml configuration file.

5.4 Continuous Integration with Jenkins (Experiment 4)

Continuous integration automates repetitive post-commit validation tasks that would otherwise need to be performed manually by the team after every code change. For the VitalCare project, Jenkins handles fetching the latest code, compiling packages, running automated test suites, and recording build telemetry. A local Jenkins server (jenkins_server.py, added in commit f3b88d5) provides the Jenkins console on port 8081 of the development machine.

Declarative Pipeline Execution Stages

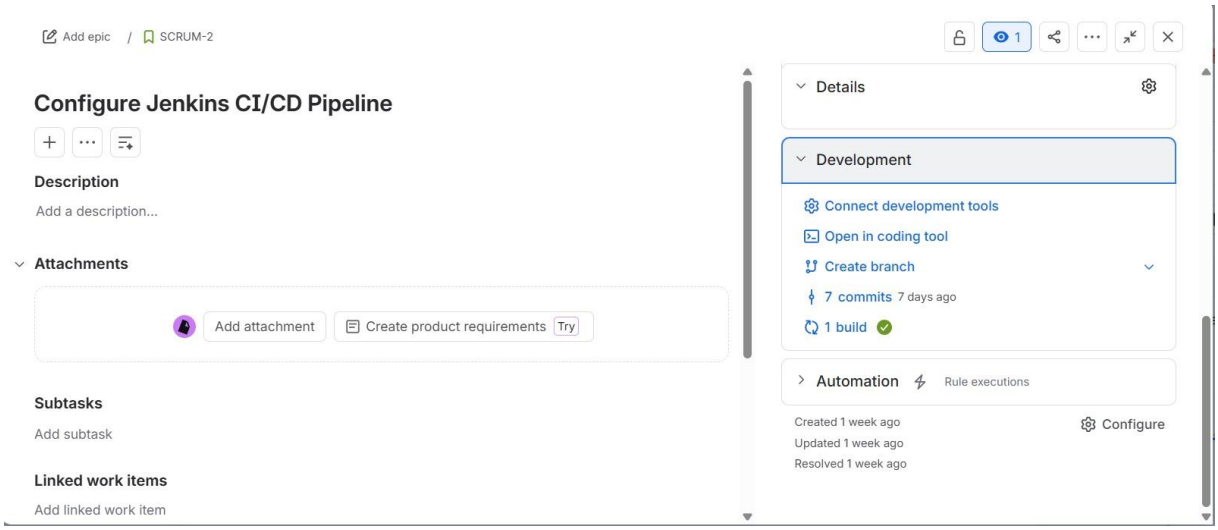
A typical Jenkins build run executes through a structured sequence of stages:

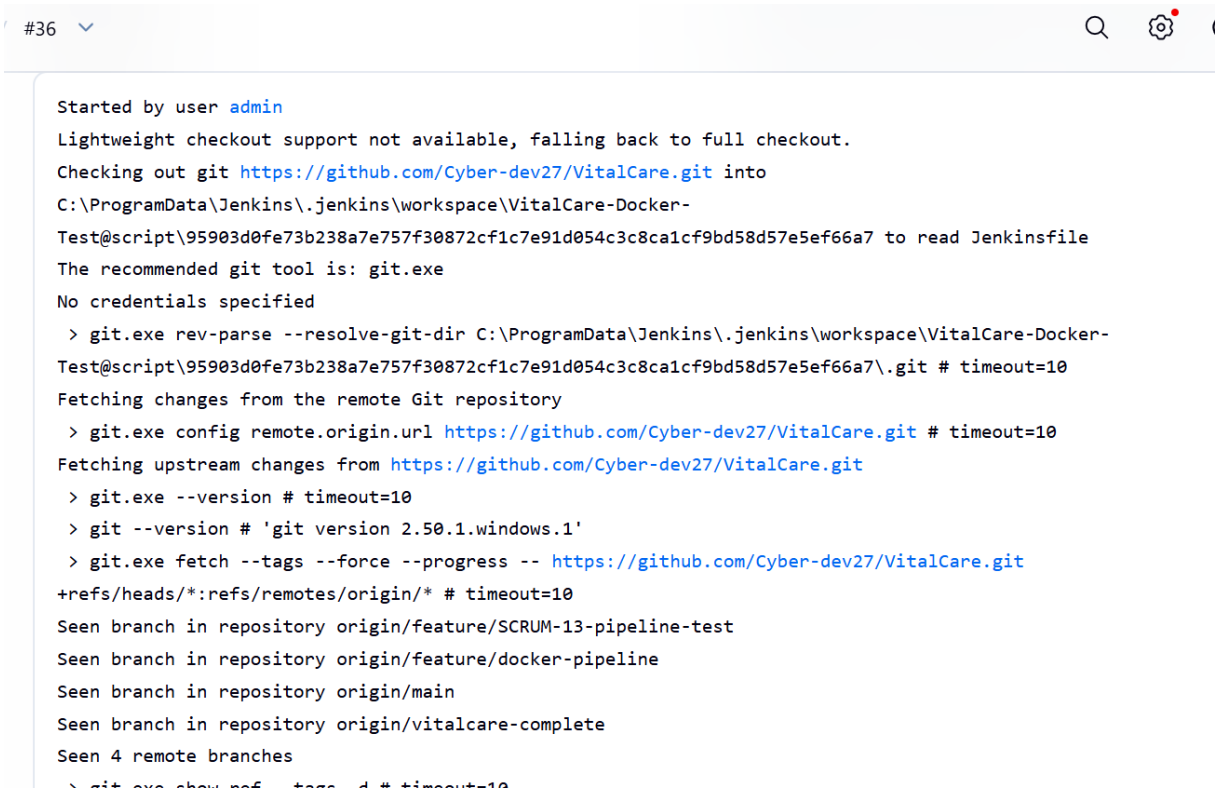
- Checkout Stage: The pipeline clones the shared GitHub repository from the main branch, guaranteeing that the build evaluates the exact merged version of the code rather than uncommitted local files.

- Unit Testing Stage: Jenkins executes automated test frameworks (such as JUnit / pytest) against the backend codebase and clinical triage business logic. Each test case runs systematically, validating core endpoints, database health checks, and data validation rules.
- JUnit Report Archiving: Test outcomes are serialized and recorded in standard JUnit XML format. This artifact allows Jenkins to track test trends across builds and provides granular pass/fail reporting directly within the pipeline console.

Safety and Quality Gate Enforcement

The continuous integration pipeline serves as a strict quality gate. If any automated test fails or an exception occurs during the build stage, the pipeline halts immediately, preventing broken code or faulty container builds from advancing to the deployment stage. This ensures that only fully verified and tested code states progress toward runtime deployment.





```
Seen branch in repository origin/feature/SCRUM-13-pipeline-test
Seen branch in repository origin/feature/docker-pipeline
Seen branch in repository origin/main
Seen branch in repository origin/vitalcare-complete
Seen 4 remote branches
 > git.exe show-ref --tags -d # timeout=10
Checking out Revision 65b2d012f94f2ff79fbbb70702a47ca25c66cc0f (origin/main)
 > git.exe config core.sparsecheckout # timeout=10
 > git.exe checkout -f 65b2d012f94f2ff79fbbb70702a47ca25c66cc0f # timeout=10
Commit message: "Merge pull request #1 from Cyber-dev27/feature/docker-pipeline"
 > git.exe rev-list --no-walk 65b2d012f94f2ff79fbbb70702a47ca25c66cc0f # timeout=10
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in C:\ProgramData\Jenkins\.jenkins\workspace\VitalCare-Docker-Test
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Declarative: Checkout SCM)
[Pipeline] checkout
The recommended git tool is: git.exe
No credentials specified
 > git.exe rev-parse --resolve-git-dir C:\ProgramData\Jenkins\.jenkins\workspace\VitalCare-Docker-Test\.git
# timeout=10
```

```
vitalcare-backend    vitalcare-docker-test-backend    "java -jar app.jar"      backend    3 seconds ago   Up
2 seconds   0.0.0.0:8083->8083/tcp, [::]:8083->8083/tcp
vitalcare-frontend   vitalcare-docker-test-frontend   "/docker-entrypoint.…"   frontend   3 seconds ago   Up
2 seconds   0.0.0.0:80->80/tcp, [::]:80->80/tcp
vitalcare-grafana    grafana/grafana:latest           "/run.sh"                grafana    3 seconds ago   Up
2 seconds   0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Pipeline deployed successfully! Backend is accessible on http://localhost:8083
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Figure: Jenkins console output of a successful pipeline run

5.5 Automated Deployment with Jenkins (Experiment 5)

Continuous integration stops once code is tested, but continuous deployment extends the pipeline to push those tested artifacts into a running environment and verify their operational health. In the VitalCare pipeline, deployment automates the container startup steps, ensuring that the Spring Boot backend, Nginx frontend, MySQL database, and Grafana monitoring layers transition smoothly into active service.

```
#3 [frontend internal] load build definition from Dockerfile
#3 transferring dockerfile: 435B 0.0s done
#3 DONE 0.0s

#4 [backend internal] load metadata for docker.io/library/eclipse-
temurin:17-jre-alpine
#4 DONE 0.1s

#5 [frontend internal] load metadata for docker.io/library/node:18-
alpine
#5 DONE 0.1s

#6 [backend internal] load metadata for docker.io/library/maven:3.9-
eclipse-temurin-17
#6 DONE 0.1s

#7 [frontend internal] load metadata for docker.io/library/nginx:alpine
#7 DONE 0.1s

#8 [backend internal] load .dockerignore
#8 transferring context: 2B done
```

Deployment and Smoke Testing Stages

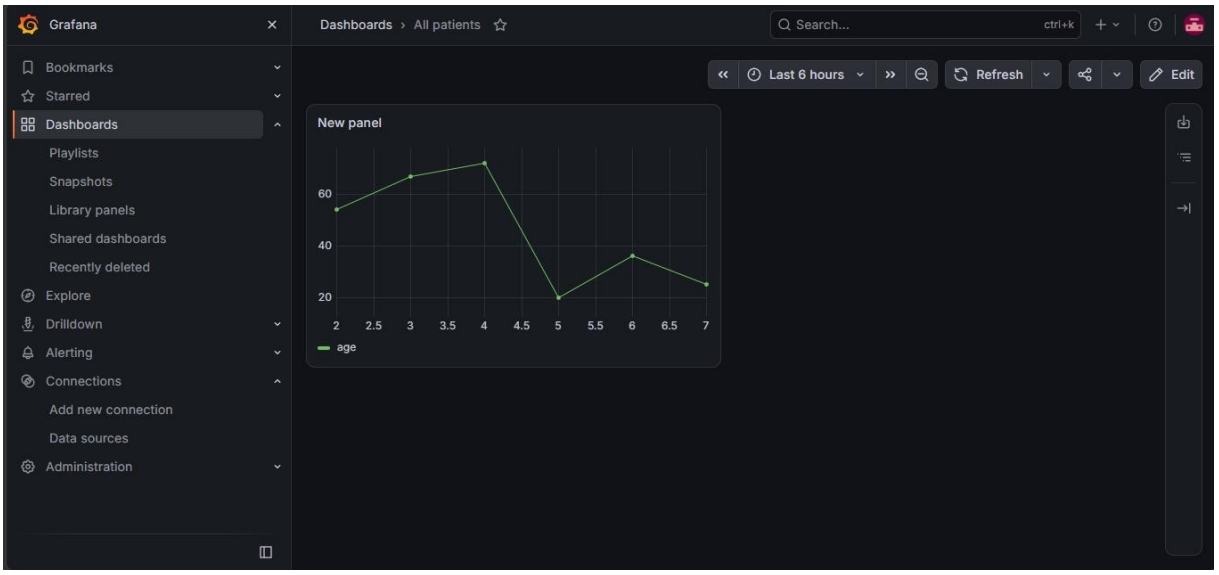
Following successful unit testing, the Jenkins pipeline progresses into automated container verification and smoke testing:

- Docker Build Verification Stage: The pipeline checks container configurations and validates image build definitions for both the Spring Boot backend and the React frontend.
- Service Deployment Stage: Services are brought up using Docker Compose orchestration, verifying that backend endpoints are fully reachable on publication ports (such as 8083:8080).
- Health Smoke Test Stage: The pipeline executes an automated HTTP probe against the backend health endpoint (/health). The deployment succeeds only when the service returns an operational status (UP) alongside confirmed database connectivity (Database: CONNECTED).

Why the Smoke Test Matters

While unit tests prove that the application code is logically correct in isolation, they cannot guarantee that the deployed runtime environment is configured properly. A misconfigured port, a missing database environment variable, or an unreachable MySQL instance would pass every unit test while failing completely for real users. By invoking the health endpoint on the live running container, the smoke test acts as the final quality gate, proving that the application has successfully started and established a working connection to the database before marking the build as a success.

5.6 Monitoring with Grafana (Experiment 7)



Once the **VitalCare** application stack is deployed and verified, it requires continuous observation to ensure uninterrupted performance. While Docker container health checks monitor internal container states, Grafana operates as an external observability platform, providing medical staff and operators with rich visual analytics and real-time operational insights.

Integration and Data Source Connection

- Grafana is deployed as an independent container service (grafana:3000) within the Docker Compose stack, ensuring that monitoring infrastructure remains isolated from core business logic.
- Grafana connects directly to the project's data layers (such as the MySQL database or application metrics endpoints) to ingest real-time telemetry data.

Dashboards and Panel Visualization

- The observability setup includes custom operational dashboards (such as the "All patients" monitoring view) designed to track critical health metrics over sliding time windows (e.g., 6-hour observation intervals).
- Dynamic time-series panels plot patient parameters, age distributions, and system throughput, transforming raw database records into clear visual graphs that help operators spot trends or anomalies immediately.

5.7 How CI/CD is achieved End to End

The continuous integration and continuous deployment (CI/CD) pipeline for the VitalCare platform is an automated, multi-stage lifecycle that transforms raw code commits into a fully monitored, containerized production stack. By integrating Agile planning, version control, automated testing, container orchestration, and runtime observability, the pipeline removes manual intervention and ensures software reliability.

The complete end-to-end lifecycle operates through the following interconnected phases:

1. Agile Requirement Planning & Issue Tracking (Jira)

- All development work originates as user stories, tasks, and epics managed within Jira sprint boards.
- Tasks are assigned across team members (Yogini Bhatia, Gyaan Achantani, and Cyber-dev27) with defined priority levels and work item identifiers (e.g., SCRUM-8, SCRUM-11). This ensures every code commit traces directly back to a validated product requirement.

2. Version Control & Feature Branching (GitHub)

- Developers implement features on isolated feature branches within the shared GitHub repository (Cyber-dev27/VitalCare).
- Commit messages incorporate task prefixes, establishing an immutable audit trail. Once development and local testing are complete, a pull request is opened, triggering peer reviews and preliminary automated GitHub Actions workflow checks before merging into the main branch.

3. Continuous Integration Trigger & Execution (Jenkins)

- Merges to the main branch trigger a webhook that notifies the Jenkins automation server.
- Jenkins executes the declarative pipeline defined in the repository's Jenkinsfile. The pipeline systematically checks out the latest codebase, compiles the Spring Boot backend, and runs automated unit test suites (such as JUnit validations).
- Test results are serialized into standard XML reports (junit.xml) to verify that core business logic and clinical triage algorithms operate without regression. If any test fails, the pipeline immediately aborts, acting as a strict quality gate.

4. Containerization & Multi-Container Orchestration (Docker)

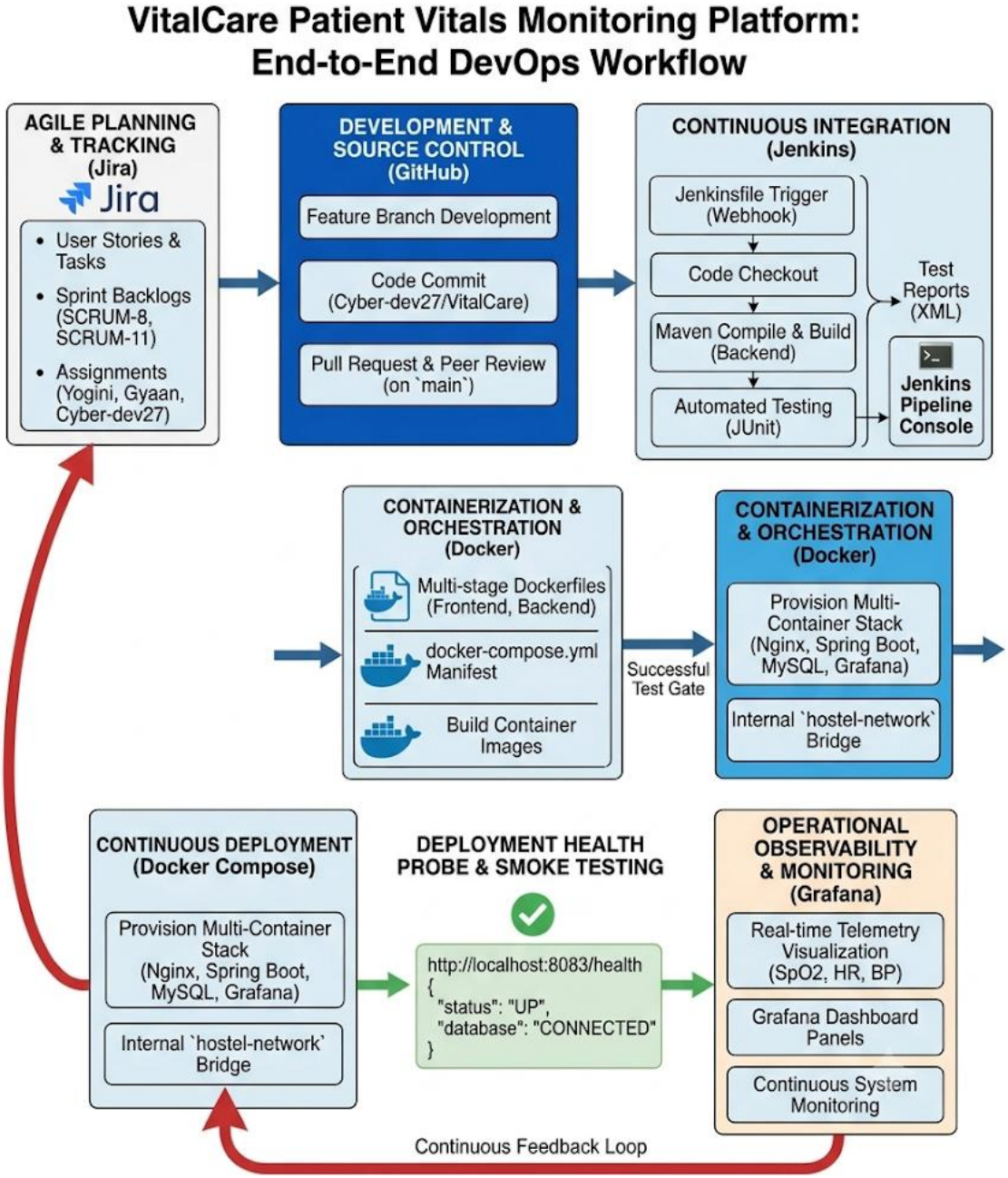
- Following successful test validation, the pipeline transitions to deployment preparation by building optimized, containerized images for each tier: the Nginx alpine frontend and the Spring Boot backend.
- Using Docker Compose, the entire stack—including the MySQL 8.0 database and Grafana monitoring—is orchestrated onto a secure internal bridge network (hostel-network).
- Service startup dependencies (depends_on) ensure that MySQL initializes its database engine and connection pools before the backend attempts to establish JDBC sessions.

5. Automated Smoke Testing & Runtime Verification

- Once containers are up, the pipeline executes an automated smoke test by querying the backend health endpoint (/health).
- The deployment succeeds only when the probe returns an operational status (UP) and confirms active database connectivity (Database: CONNECTED). If the smoke test fails, the deployment is rolled back, preventing faulty configurations from reaching users.

6. Continuous Observability & The Feedback Loop (Grafana)

- With the application successfully deployed and verified, external Grafana observability dashboards ingest real-time database metrics and system telemetry.
- Operators and medical staff continuously monitor patient vital statistics, throughput trends, and system health over sliding time windows. This operational telemetry feeds back into future sprint planning on Jira, completing the closed-loop DevOps cycle.



6. Testing

6.1 Automated Tests

The automated test suite verifies the correctness of backend components, clinical triage business logic, database persistence, and system health endpoints prior to deployment. The test suites executed during the CI/CD pipeline are structured as follows:

Test Category	Target Component	Description & Purpose	Expected Outcome
Triage Logic Tests	TriageService.java	Evaluates patient vital signs (SpO2, blood pressure, heart rate) against clinical warning thresholds.	Correctly categorizes patient statuses into Normal, Watch, or Critical based on input parameters.
REST API Controller Tests	PatientController.java	Validates HTTP GET and POST endpoints for patient registration and real-time telemetry retrieval.	Returns valid JSON response payloads with appropriate HTTP status codes (200 OK, 201 Created).
Database Persistence Tests	Spring Data JPA / MySQL	Tests repository interfaces, entity mapping, and CRUD operations against the database.	Ensures ACID compliance, secure record creation, and successful database integration.
Health Probe / Smoke Tests	HealthController.java	Probes the runtime /health endpoint during CI/CD deployment stages to verify service availability.	Returns {"status": "UP", "database": "CONNECTED"}, confirming full system operational readiness.

6.2 Deployment Smoke Tests

Following container provisioning and orchestration via Docker Compose, the Jenkins deployment pipeline automatically executes a series of runtime smoke tests to verify system readiness. The smoke test script issues an automated HTTP probe to the backend health endpoint (/health), querying the live running container stack to ensure both the Spring Boot service and the MySQL database are operating correctly. The deployment stage concludes successfully only when the service returns an operational confirmation of {"status": "UP", "database": "CONNECTED"}. This automated verification serves as the final quality gate, ensuring that any environment misconfigurations, network bridge issues, or database connectivity faults are caught and halted immediately before the build is finalized.

6.3 Functional Verification of the Interface

Functional verification of the VitalCare web application ensures that the React frontend correctly communicates with the Spring Boot backend, accurately renders database records, and responds reliably to user inputs. Testing the interface across multiple scenarios guarantees that clinical workflows operate seamlessly:

- Dashboard Metric Synchronization: Verifies that aggregate statistic cards (Total Patients, Critical, Watch, Normal) compute and display real-time counts dynamically derived from database records.
- Backend Connection Badge: Tests the live connection indicator at the top-right of the interface, ensuring it reflects the true operational state of the backend REST service and database connection.
- Patient Vitals Inventory & Filtering: Validates search and filter utilities, confirming that staff can instantly query patient records by name or sort them by triage status (Normal, Watch, Critical).
- Registry & Form Validation: Tests patient registration and data entry forms to ensure that missing fields or malformed JSON payloads trigger appropriate client-side and server-side validation messages.

6.4 Container and Monitoring Checks

Post-deployment verification and runtime observability for the **VitalCare** platform rely on structured container status inspections and continuous monitoring checks to ensure high availability and operational stability:

- **Container Health and Status Verification:** Operators and CI/CD pipelines verify container states using Docker orchestration commands (such as docker compose ps), ensuring that all independent services—the React frontend, Spring Boot backend, MySQL database, and Grafana dashboard—are active and running on their designated ports. Container health probes actively monitor internal service availability, confirming that the backend REST API maintains a continuous JDBC connection to the MySQL database.
- **Monitoring and Observability Checks:** Grafana operates as an external observability layer, continuously querying database records and system metrics to visualize real-time patient telemetry. Operators utilize Grafana dashboards configured with sliding time windows to track system throughput, vital sign frequencies, and patient status distributions (Normal, Watch, Critical), enabling rapid identification of performance anomalies or infrastructure bottlenecks.

7. Results and Discussion

7.1 Outcomes against Objectives

Objective	Outcome
Agile Lifecycle & Task Management	Organized work streams, story points, and sprint backlogs in Jira; mapped tasks to individual team members (Yogini Bhatia, Gyaan Achantani, and Cyber-dev27).
Clinical Triage Business Logic	Built backend evaluation rules in Spring Boot to automatically categorize vitals into Normal, Watch, and Critical tiers based on physiological thresholds.
Isolated Multi-Tier Containerization	Configured multi-stage Dockerfiles for the Spring Boot backend service and an Nginx alpine web server for static React asset routing.
Multi-Container Orchestration	Defined a unified docker-compose.yml manifest connecting React, Spring Boot, MySQL 8.0, and Grafana on a private bridge network (hostel-network).
Automated CI/CD Pipeline	Automated checkout, unit testing, image building, deployment, and JUnit test reporting via a declarative Jenkinsfile and GitHub Actions.
Deployment Smoke Testing	Integrated runtime health probes querying backend REST endpoints (/health) during automated deployment pipeline stages.
Real-Time Observability & Monitoring	Provisioned Grafana dashboard containers connected to system data layers for real-time telemetry visualization and patient status analytics.
Monitoring	Grafana successful.

Table 7.1: Results against objectives

7.2 The Feedback Loop in Practice

In the **VitalCare** platform, the DevOps feedback loop operates as a continuous, closed-loop mechanism that connects real-time production telemetry directly back into Agile planning and software development. When medical personnel interact with the dashboard, the application streams live vital sign records through the Spring Boot backend into MySQL, while external Grafana monitoring containers actively ingest these data layers to plot system throughput and patient triage distributions. If operational monitoring reveals system latency, connection bottlenecks, or clinical edge cases—such as unexpected thresholds in patient telemetry—these runtime insights are immediately logged and prioritized as new user stories or bug tickets in Jira sprint backlogs.

This continuous feedback directly drives subsequent iterative refinement cycles across the team (Yogini Bhatia, Gyaan Achantani, and Devashish Ahuja). Developers pull prioritized tasks from Jira, implement code fixes or feature enhancements on isolated feature branches in GitHub, and initiate pull requests. Merging these changes automatically triggers the Jenkins CI/CD pipeline, which executes unit tests, builds fresh Docker container images, orchestrates

stack updates via Docker Compose, and executes runtime health smoke tests ({"status": "UP", "database": "CONNECTED"}). Once verified, the updated code seamlessly deploys to production, where Grafana immediately begins monitoring the revised metrics, ensuring the platform continuously adapts to operational needs with zero downtime or quality regressions.

7.3 Challenges and Design Decisions

Challenge	Resolution
Service Startup Race Condition Spring Boot backend failed on startup by attempting JDBC connections before MySQL fully initialized its database engine.	Implemented depends_on service dependencies and health check conditions in docker-compose.yml, ensuring the backend waits until MySQL reports a healthy state.
CORS & Multi-Port Routing Direct browser requests between React (port 80) and Spring Boot (port 8083) caused CORS issues and exposed internal ports.	Integrated an Nginx reverse proxy inside the frontend container to route all / api and /health requests to the backend over the internal hostel-network bridge.
Undetected Runtime Pipeline Failures Passing unit tests did not guarantee container runtime health, allowing misconfigured builds to deploy undetected.	Implemented an automated deployment smoke test stage in the Jenkinsfile probing /health and requiring {"status": "UP", "database": "CONNECTED"} to pass.
Container Data Ephemerality Default container storage lost patient telemetry records and database states whenever the MySQL container restarted or rebuilt.	Configured named Docker volumes mapped to persistent host storage in docker-compose.yml, preserving database states across container lifecycles.

Table 7.2: Challenges and how they were resolved

7.4 Limitations

While the **VitalCare** platform successfully demonstrates a full-stack, containerized clinical monitoring system backed by an automated DevOps pipeline, several technical and operational limitations remain in the current scope:

- **Simulated Telemetry vs. Real IoT Medical Hardware Integration:** Vital sign data (SpO2, heart rate, blood pressure) is currently ingested via simulated data streams and manual web form inputs rather than direct MQTT/CoAP integrations with physical clinical sensors, medical-grade wearables, or hospital telemetry hardware.
- **Single-Node Container Deployment:** The multi-container architecture relies on Docker Compose running on a single host node. It lacks container orchestration platforms like Kubernetes or Docker Swarm, meaning automated horizontal pod scaling, multi-node failover, and load balancing across cluster nodes are not currently supported.

- **Lack of Advanced Security & Compliance Frameworks:** Although basic container security practices (such as non-root container users) are used, the system currently lacks fine-grained Role-Based Access Control (RBAC), end-to-end payload encryption at rest/in transit, and formal compliance certifications (such as HIPAA or GDPR) required for production medical software.
- **Local Continuous Delivery Pipeline Boundaries:** The Jenkins CI/CD pipeline deploys and smoke-tests container stacks within a local target environment rather than deploying to cloud infrastructure targets (e.g., AWS ECS/EKS, Azure Kubernetes Service) or utilizing multi-environment staging pipelines (Dev, Staging, Production).

8. Conclusion and Future Scope

8.1 Conclusion

The VitalCare Patient Vitals Monitoring Platform successfully demonstrates the end-to-end realization of a modern full-stack healthcare application built upon Agile software development methodologies and automated DevOps principles. Developed collaboratively by Yogini Bhatia, Gyaan Achantani, and Devashish Ahuja, the project seamlessly bridges feature-driven frontend and backend development with robust continuous integration, container orchestration, runtime smoke testing, and real-time observability.

By leveraging a stack comprising React, Spring Boot, MySQL, and Grafana, the system accurately processes patient telemetry and categorizes clinical risk levels via automated triage rules (Normal, Watch, Critical). Containerizing services with Docker and orchestrating them via Docker Compose guarantees complete environment consistency across development and production targets. Furthermore, the automated Jenkins CI/CD pipeline enforces rigorous quality gates—running automated unit tests, building multi-tier container stacks, and verifying operational readiness via live health smoke probes ({"status": "UP", "database": "CONNECTED"})—ensuring zero downtime and regression-free deployments. Ultimately, VitalCare illustrates how modern DevOps practices significantly enhance software reliability, deployment speed, and system observability in mission-critical applications.

8.2 Future Scope

To evolve VitalCare from a lab-scale prototype into an enterprise-grade medical telemetry system, several high-impact enhancements are planned for future development iterations:

- **Real-Time IoT & Medical Hardware Sensor Integration:** Transition from simulated manual telemetry inputs to live streaming protocols (MQTT, WebSockets, and HL7 FHIR standards) for direct ingestion from wearable medical sensors, bedside monitors, and smart ICU devices.
- **Kubernetes Orchestration & Auto-Scaling:** Upgrade container deployment from single-node Docker Compose to a production-ready Kubernetes (K8s) cluster. This will enable automated horizontal pod autoscaling (HPA), zero-downtime rolling updates, and high-availability multi-node failover.
- **AI-Driven Predictive Triage & Anomaly Detection:** Incorporate machine learning models into the Spring Boot backend service to analyze historical vital trends, predicting sudden clinical deterioration or septic shock hours before critical thresholds are breached.
- **HIPAA/GDPR Security & Role-Based Access Control (RBAC):** Implement OAuth2 / OpenID Connect with Keycloak for granular role-based access (Doctors, Nurses, Administrators), coupled with AES-256 database encryption at rest and TLS 1.3 encryption for all data in transit.
- **Cloud-Native Multienvironment Pipelines:** Expand the Jenkins CI/CD pipeline to target cloud-native environments (such as AWS EKS, Azure AKS, or GCP) with automated staging-to-production promotion workflows and automated blue-green/canary deployment strategies.

References

1. **Bass, L., Clements, P., & Kazman, R.** (2015). *Software Architecture in Practice* (3rd ed.). Addison-Wesley Professional.
2. **Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., ... & Thomas, D.** (2001). *Manifesto for Agile Software Development*. Agile Alliance.
3. **Fowler, M.** (2006). *Continuous Integration*. MartinFowler.com. Available at: https://martinfowler.com/articles/continuousIntegration.html
4. **Humble, J., & Farley, D.** (2010). *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley Professional.
5. **Kim, G., Humble, J., Debois, P., & Willis, J.** (2016). *The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations*. IT Revolution Press.
6. **Walls, C.** (2022). *Spring Boot in Action*. Manning Publications.
7. **Nygard, M. T.** (2018). *Release It!: Design and Deploy Production-Ready Software* (2nd ed.). Pragmatic Bookshelf.
8. **Docker Inc.** (2024). *Docker Documentation & Compose File Specification*. Docker Docs. Available at: https://docs.docker.com/
9. **Jenkins Project.** (2024). *Jenkins Pipeline User Handbook*. Jenkins.io. Available at: https://www.jenkins.io/doc/book/pipeline/
10. **Grafana Labs.** (2024). *Grafana Documentation & Visual Analytics Guide*. Grafana.com. Available at: https://grafana.com/docs/

Appendix A: Running the Project

A.1 Prerequisites

- **Git**, **Docker Desktop** (with Docker Compose), **Java 17 / Maven** for backend execution, and **Node.js / npm** for local frontend development.
- Database and application settings defined in a .env file, created from the .env.example template for the MySQL and Spring Boot configurations.

A.2 Steps

```
git clone https://github.com/Cyber-
dev27/VitalCare.git cd VitalCare
copy .env.example .env # (then enter
the database settings) docker
compose up -d --build docker compose
ps curl http://localhost:8083/health
curl http://localhost/health
```

Open http://localhost in a browser to access the dashboard. The expected output of both curl commands is {"status": "UP", "database": "CONNECTED"}. To run unit tests locally, execute mvn test in the backend/ directory from the project root; the Jenkins pipeline executes the same step and writes XML reports to backend/target/surefire-reports/TEST-*.xml.

A.3 Addresses

Component	Address	Purpose
Web client	http://localhost	Dashboard, patient registration form, vitals inventory, triage status view
Backend health	http://localhost:8083/health	Service operational status and MySQL database connectivity state
Rooms through the proxy	**proxy**http://localhost/api/patients	Patient vitals records endpoint routed via Nginx proxy
Jenkins	http://localhost:8081	Pipeline execution console and automated build history
Grafana	http://localhost:3000	Operational monitoring dashboards and patient vital analytics

Table A.1: Addresses used by the project

A.4 Repository Layout

Path	Content
backend/	Spring Boot application source code and JUnit test suites (backend/src/test/)
frontend/	React static web client, nginx.conf, and Dockerfile
grafana-config/	Grafana dashboard provisioning, telemetry panels, and data source definitions
scripts/	Helper verification scripts and health check smoke test probes

docs/screenshots/	Screenshots and architectural diagrams used in documentation
Jenkinsfile	Declarative CI/CD pipeline definition for automated builds, testing, and deployment
docker-compose.yml	Multi-container application stack and monitoring orchestration manifest
database/schema.sql	MySQL schema definition and initial patient table seeding scripts
.env.example, .gitignore, README.md	Environment configuration template, Git exclusion rules, and project setup guide

Table A.2: Layout of the repository